Bosch Global Software Technologies

PDF Document Extraction Pipeline

Fully Local, End-to-End Structured Data Extraction from PDF Documents

No proprietary APIs · No cloud dependency · Runs entirely on your machine

Author Aman Pant (MTQ3KOR)

Department ERD – Engineering Research & Development

Organization Bosch Global Software Technologies

Role Intern 2026

Project Table Extraction – Mainstream

April 2026

Acknowledgement

I would like to express my sincere gratitude to **Bosch Global Software Technologies** for providing me the opportunity to work on this project as part of the ERD Internship 2026 program.

I am thankful to my mentors and the team at Bosch for their continuous guidance, support, and encouragement throughout the development of this pipeline. Their expertise in document processing and data engineering helped shape the direction and quality of this work.

I also acknowledge the open-source communities behind GLM-OCR, PP-DocLayout-V3, Qwen2.5, Ollama, and vLLM, whose tools made this fully local, privacy-preserving pipeline possible.

Aman Pant (MTQ3KOR)
ERD Intern 2026
Bosch Global Software Technologies

Contents

Abstract

This report documents the design, implementation, and evaluation of a fully local, end-to-end pipeline for extracting structured JSON data from commercial PDF documents such as purchase orders and shipping bills.

The pipeline operates in two stages. **Stage 1** performs OCR and raw extraction using GLM-OCR (0.9B parameters) with PP-DocLayout-V3 for layout analysis, producing Markdown and structured JSON outputs. **Stage 2** applies a hybrid AI and rule-based extraction engine that maps OCR outputs to schema-aligned JSON—using Qwen2.5:32b via Ollama for free-text fields and deterministic HTML table parsing for structured tables.

A **Streamlit-based evaluation frontend** allows side-by-side comparison of extraction results against ground truth and GPT-based baselines, generating color-coded Excel reports with field-level MATCH, MISMATCH, NEAR_MATCH, and MISSING_KEY analysis.

The system requires zero proprietary APIs, runs entirely on local hardware, and achieves **85–90% match rates** while maintaining **100% accuracy** on rule-based table extraction. On purchase orders, the local pipeline **outperforms GPT by 26 percentage points**. Adding new document types requires zero changes to any core pipeline file.

Introduction

2.1 Problem Statement

Commercial documents—purchase orders, shipping bills, invoices—arrive as PDF files, but downstream enterprise systems require structured data. Manually keying in hundreds of fields across dozens of pages is:

- **Slow** – a single purchase order with 24 line items across 14 pages takes 30–60 minutes to key in manually.
- **Error-prone** – manual data entry introduces transcription errors, especially in numeric fields like quantities, prices, and dates.
- **Expensive** – at scale, this requires dedicated data entry teams costing significant operational overhead.
- **Privacy-sensitive** – cloud-based extraction services require uploading confidential commercial documents containing pricing, supplier terms, and trade secrets to third-party servers.

Existing solutions either rely on proprietary cloud APIs (raising data privacy concerns for sensitive commercial documents), require extensive per-document training data, or fail on complex table layouts that span multiple pages—a common pattern in commercial documents.

2.2 Aim of the Project

To build a **fully local, end-to-end pipeline** that takes a raw PDF document as input and produces a clean, schema-aligned JSON file as output—without any cloud dependency, proprietary APIs, or API keys—while providing an intuitive evaluation interface for quality assessment.

2.3 Objectives

1. Deliver **fast, reliable, and user-friendly** extraction of structured data from complex commercial PDF documents with minimal human intervention.
2. Produce clean **key–value pair JSON output** that is schema-aligned, validated, and ready for direct consumption by downstream enterprise systems.
3. Ensure **high extraction accuracy** across both free-text fields (names, dates, addresses) and structured tabular data (line items, shipment schedules, pricing).
4. Build a **fully local, privacy-preserving** solution that processes confidential documents entirely on-premises with zero cloud dependency.
5. Design a **generic and extensible** architecture that supports new document types without modifying the core extraction engine.
6. Provide an **intuitive evaluation interface** that enables non-technical users to visually assess and compare extraction quality through downloadable reports.

2.4 Scope of the Project

In Scope

- OCR extraction from PDF/image documents using GLM-OCR (0.9B params)
- Layout detection using PP-DocLayout-V3 (25 label categories)
- Structured data extraction using Qwen2.5:32b (local) + rule-based parsing
- Two document types: Purchase Order (Li & Fung) and Shipping Bill (Indian Customs)
- Streamlit evaluation frontend with 3-way comparison (GT vs MyModel vs GPT)
- Color-coded Excel report generation with per-field status tracking
- Extensible plugin architecture for new document types

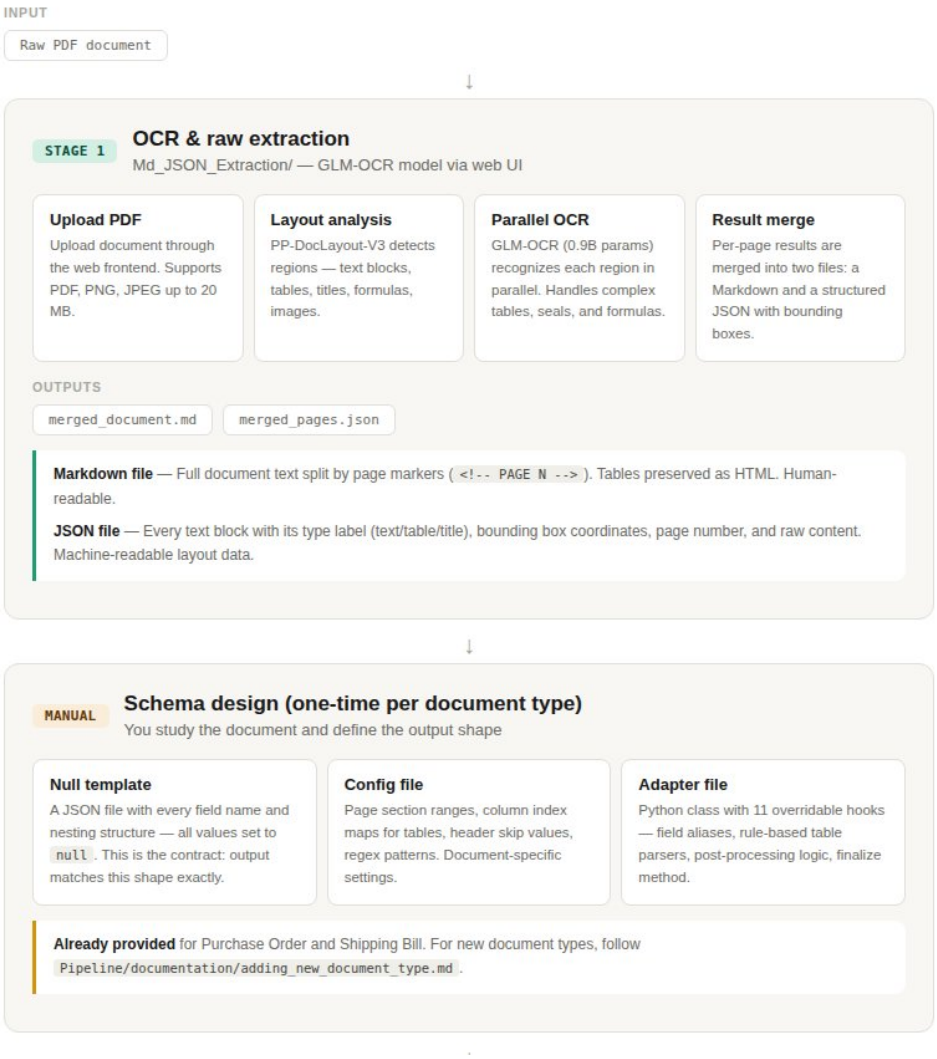
Out of Scope

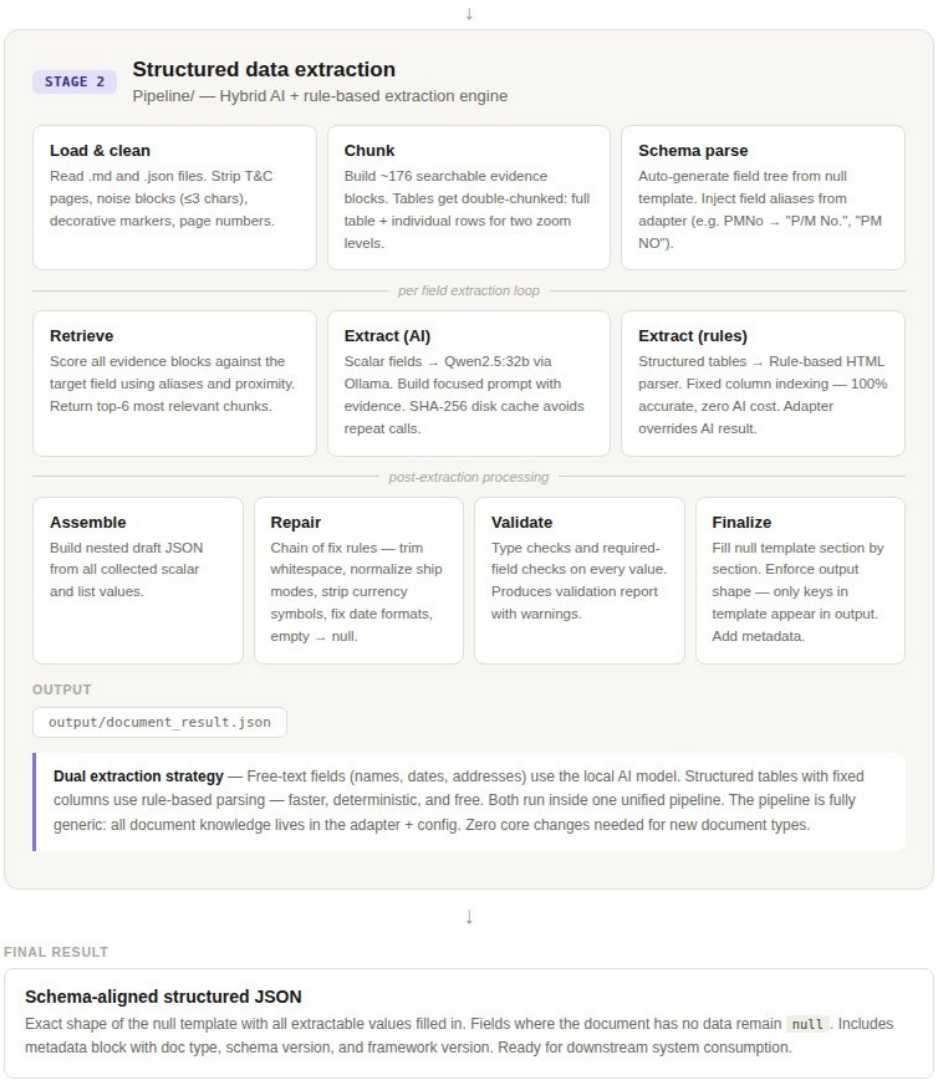
- Handwritten document recognition
- Real-time streaming extraction
- Cloud deployment or SaaS packaging
- Training or fine-tuning of OCR/LLM models
- Languages other than English

Project Description

3.1 Workflow Overview

The pipeline operates in two stages connected by a one-time manual schema design step. The overall flow is: Raw PDF → OCR Extraction → Schema Design → Structured Extraction → JSON Output → Evaluation.




STAGE 2
Structured data extraction
Pipeline/ — Hybrid AI + rule-based extraction engine
Load & clean
Read .md and .json files. Strip T&C pages, noise blocks (≤3 chars), decorative markers, page numbers.
Chunk
Build ~176 searchable evidence blocks. Tables get double-chunked: full table + individual rows for two zoom levels.
Schema parse
Auto-generate field tree from null template. Inject field aliases from adapter (e.g. PMNo → "P/M No.", "PM NO").
per field extraction loop
Retrieve
Score all evidence blocks against the target field using aliases and proximity. Return top-6 most relevant chunks.
Extract (AI)
Scalar fields → Qwen2.5:32b via Ollama. Build focused prompt with evidence. SHA-256 disk cache avoids repeat calls.
Extract (rules)
Structured tables → Rule-based HTML parser. Fixed column indexing — 100% accurate, zero AI cost. Adapter overrides AI result.
post-extraction processing
Assemble
Build nested draft JSON from all collected scalar and list values.
Repair
Chain of fix rules — trim whitespace, normalize ship modes, strip currency symbols, fix date formats, empty → null.
Validate
Type checks and required-field checks on every value. Produces validation report with warnings.
Finalize
Fill null template section by section. Enforce output shape — only keys in template appear in output. Add metadata.
OUTPUT
output/document_result.json
Dual extraction strategy — Free-text fields (names, dates, addresses) use the local AI model. Structured tables with fixed columns use rule-based parsing — faster, deterministic, and free. Both run inside one unified pipeline. The pipeline is fully generic: all document knowledge lives in the adapter + config. Zero core changes needed for new document types.
FINAL RESULT
Schema-aligned structured JSON
Exact shape of the null template with all extractable values filled in. Fields where the document has no data remain null. Includes metadata block with doc type, schema version, and framework version. Ready for downstream system consumption.

3.2 The 11-Step Extraction Pipeline

#	Stage	Description
1	Load Markdown	Split by `PAGE` markers, strip Terms & Conditions pages
2	Load JSON	Unwrap OCR blocks per page, handle double-wrapping
3	Clean Inputs	Strip noise blocks ($\leq$3 chars), page markers, decorative lines
4	Build Evidence	Merge into $\sim$176 searchable chunks; tables double-chunked (full + per-row)
5	Parse Schema	Auto-generate field tree from null template, inject field aliases
6	Extract Scalars	Per field: retrieve top-6 chunks $\rightarrow$ SHA-256 cache check $\rightarrow$ Qwen2.5 prompt
7	Extract Lists	Rule-based HTML table parser overrides AI for structured tables
8	Assemble	Build nested draft JSON from all collected scalar and list values
9	Repair	Fix whitespace, normalize dates/currency/ship modes, empty $\rightarrow$ null
10	Validate	Type checks (string, number, list) and required-field checks
11	Finalize	Fill null template section by section, enforce output shape, add metadata

Table 3.1: The 11-step extraction assembly line

3.3 Dual Extraction Strategy

The pipeline uses two extraction methods and intelligently picks the right one per field:

Method	Used For	How It Works	Accuracy	Speed
AI	Free-text fields	Retrieve chunks $\rightarrow$ build prompt $\rightarrow$ Qwen2.5	High	2–5s
Rules	Structured tables	HTML parser reads `<td>` by column index	100%	$<$100ms

Table 3.2: Dual extraction strategy comparison

3.4 Project Features

- **Fully Local & Private** – All processing happens on-premises. No document data ever leaves the machine, ensuring complete confidentiality of commercial documents.
- **Dual Extraction Engine** – AI handles free-text; rules handle tables. Best of both worlds.

- **User-Friendly Evaluation UI** – A clean Streamlit web interface lets users upload files, run comparisons, and download reports with a few clicks—no command-line knowledge required.
- **Generic Plugin Architecture** – The core pipeline never changes. Adding a new document type = 3 files + 1 line in the registry. Future-proof and maintainable.
- **Intelligent Caching** – SHA-256 disk-backed response cache ensures re-runs and debugging iterations are near-instant.
- **Color-Coded Excel Reports** – Downloadable Excel files with green/yellow/orange/red row coloring for instant visual assessment of extraction quality.
- **3-Way Comparison** – Compare Ground Truth vs Your Pipeline vs GPT side-by-side in a single report.
- **Per-Page & Overall Recall** – The evaluation engine computes field-level overlap recall both per-page and across the entire document.

3.5 Design Diagram – Layered Architecture

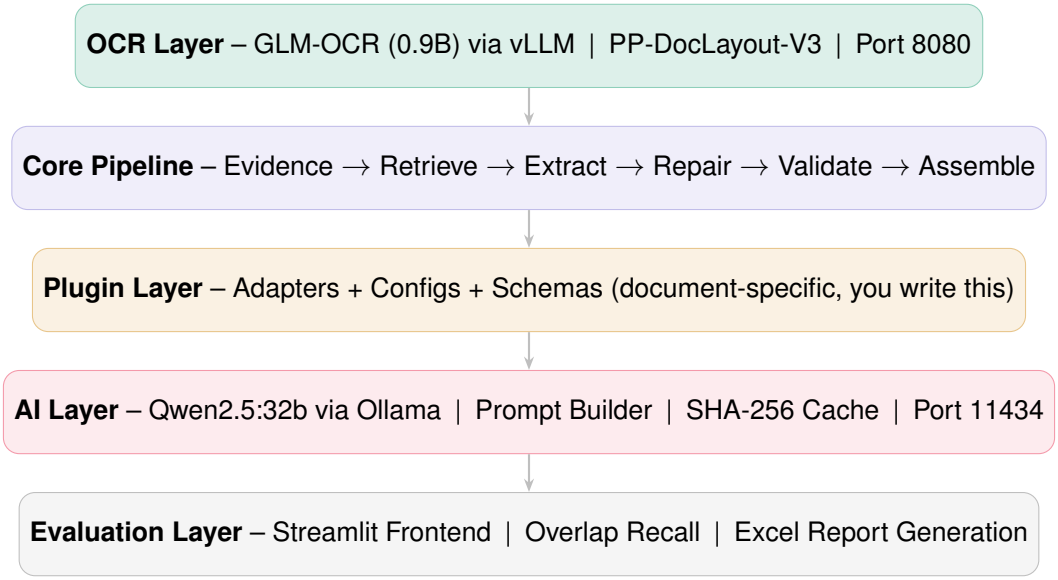


Figure 3.2: Five-layer system architecture

3.6 Design and Implementation Constraints

- All processing must run entirely offline—no external API calls permitted at any stage.
- Extraction must be deterministic and reproducible when cache is enabled.
- Output JSON must match the null template shape exactly—no extra or missing keys allowed.

- Core pipeline files must never be modified when adding new document types.
- Rule-based parsing is preferred over AI wherever column positions are fixed and known.
- The evaluation frontend must work without requiring any programming knowledge from the user.

3.7 Assumptions and Dependencies

Item	Details
GPU availability	24 GB+ VRAM recommended for serving both GLM-OCR (vLLM) and Qwen2.5 (Ollama)
Document structure	Documents follow a consistent layout within each type (column positions are fixed)
OCR quality	GLM-OCR produces accurate HTML tables with correct column counts
Ollama server	Must be running on port 11434 for AI extraction calls
vLLM server	Must be running on port 8080 for OCR inference
Python 3.12+	Required for GLM-OCR SDK and UV package manager

Table 3.3: Key assumptions and dependencies

Evaluation Frontend

The project includes a **Streamlit-based evaluation frontend** that provides a user-friendly web interface for assessing extraction quality. No command-line knowledge is required—everything is point-and-click.

4.1 Frontend Architecture

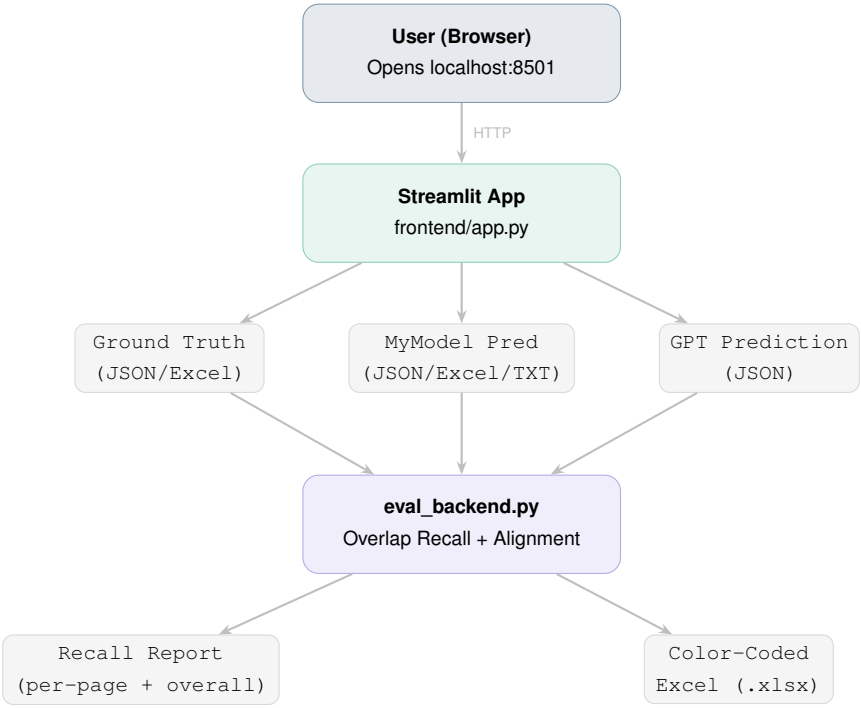


Figure 4.1: Evaluation frontend – component diagram

4.2 User Interface

The frontend presents a clean, branded interface with the Bosch logo, a dark sidebar for file uploads, and action buttons for running evaluations.

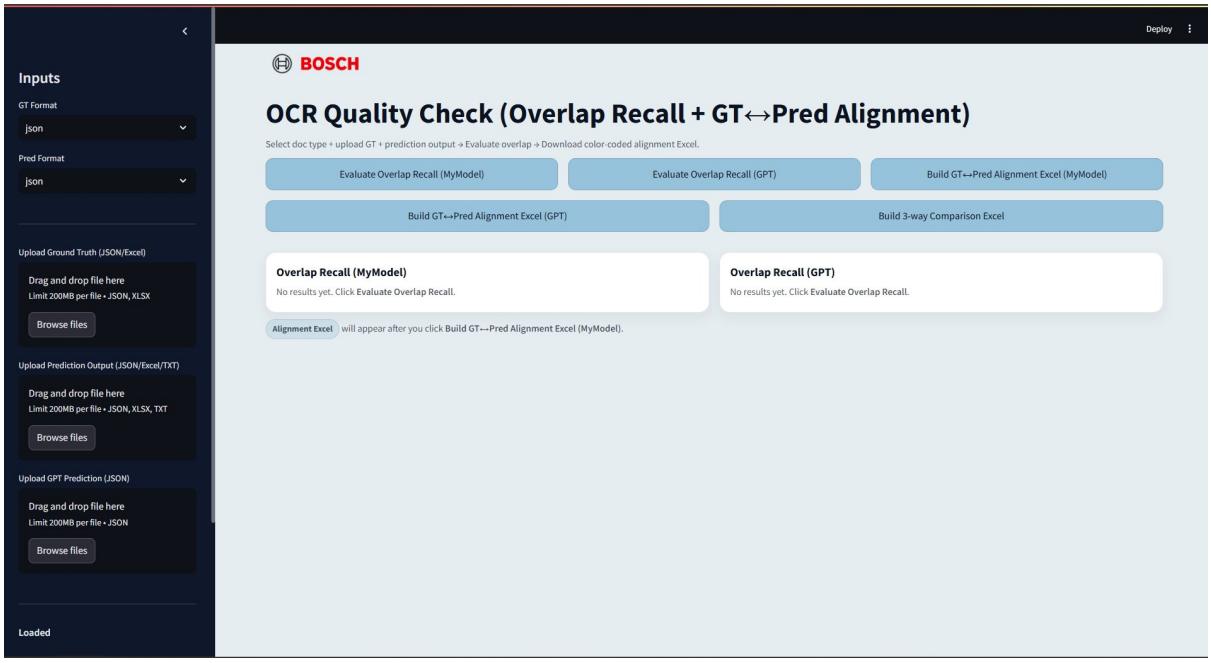


Figure 4.2: Streamlit evaluation frontend – main interface

4.3 Input Files

The user uploads **three files** through the sidebar:

Upload Slot	Format	What It Contains
Ground Truth	JSON or Excel	The expected correct values for every field in the document
MyModel Prediction	JSON, Excel, or TXT	The structured JSON output produced by our Pipeline (Stage 2)
GPT Prediction	JSON	The extraction output produced by GPT for comparison

Table 4.1: Three input files for evaluation

4.4 Evaluation Actions

Five action buttons are available:

1. **Evaluate Overlap Recall (MyModel)** – Computes per-page and overall field-level recall for the local pipeline output.
2. **Evaluate Overlap Recall (GPT)** – Same metric computed for the GPT extraction output.
3. **Build GT↔Pred Alignment Excel (MyModel)** – Generates a color-coded Excel file comparing ground truth against the local pipeline.
4. **Build GT↔Pred Alignment Excel (GPT)** – Same Excel report for the GPT output.
5. **Build 3-Way Comparison Excel** – A single Excel file comparing Ground Truth vs MyModel vs GPT side-by-side.

4.5 Excel Report Output

The generated Excel report provides a **field-by-field comparison** with five columns: GT Key, Pred Key, GT Value, Pred Value, and Status. Each row is color-coded based on the match status.

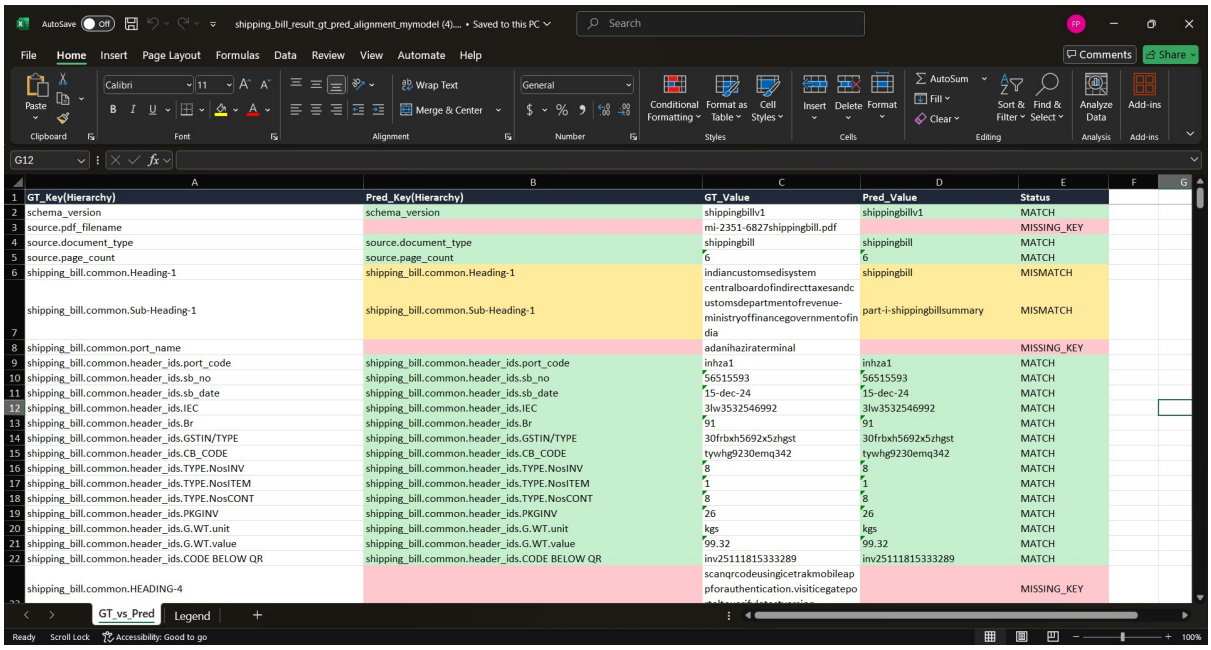
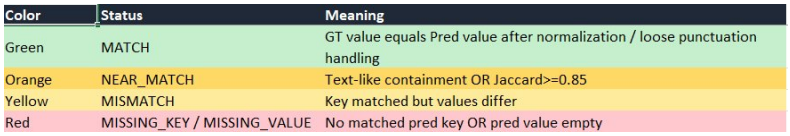


Figure 4.3: Color-coded Excel alignment report – GT vs Predicted values

4.6 Status Legend

Every field in the comparison is assigned one of four statuses:



Color	Status	Meaning
Green	MATCH	GT value equals Pred value after normalization / loose punctuation handling
Orange	NEAR_MATCH	Text-like containment OR Jaccard>=0.85
Yellow	MISMATCH	Key matched but values differ
Red	MISSING_KEY / MISSING_VALUE	No matched pred key OR pred value empty

Figure 4.4: Status legend for the Excel comparison report

Color	Status	Meaning
Green	MATCH	GT value equals Pred value after normalization and loose punctuation handling
Orange	NEAR_MATCH	Text-like containment OR Jaccard token similarity ≥ 0.85
Yellow	MISMATCH	Key was matched but the extracted value differs from ground truth
Red	MISSING_KEY	No matching prediction key found, or prediction value is empty

Table 4.2: Detailed status definitions

Project Life Cycle

5.1 Development Methodology

The project followed an **iterative, problem-decomposition approach** inspired by Agile principles. Rather than treating the entire pipeline as a monolithic problem, the core challenge was broken down into **three independent sub-problems**, each tackled as a separate research-and-build sprint. This allowed rapid experimentation within each sub-problem while maintaining a clear integration path.

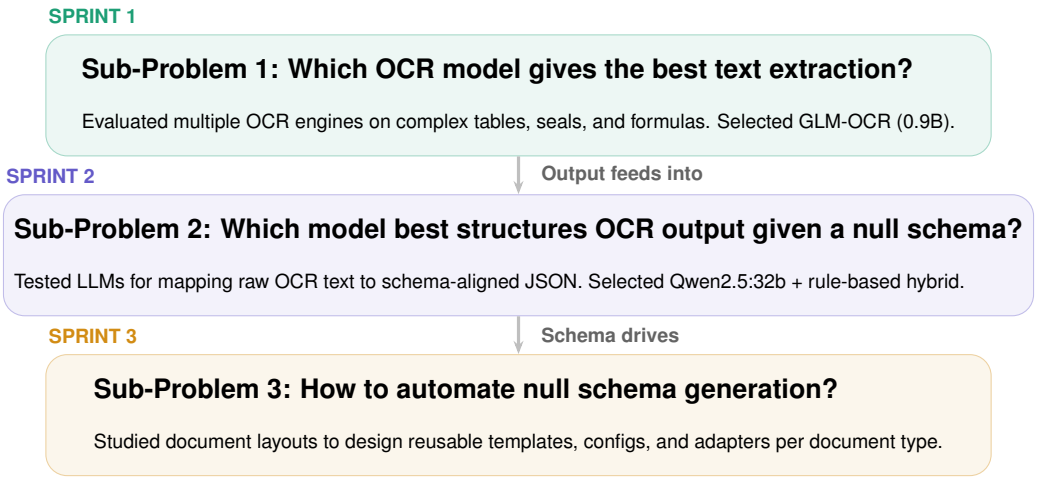


Figure 5.1: Problem decomposition into three independent sprints

5.2 Why This Approach

Each sub-problem was **independently testable**—the OCR model could be evaluated on its own using benchmark documents before any structuring pipeline existed. The structuring model could be tested with manually prepared OCR outputs before the OCR stage was finalized. This decoupling reduced risk and allowed parallel experimentation.

Within each sprint, the workflow was iterative:

1. **Research** – Survey available models and tools for the sub-problem
2. **Prototype** – Build a minimal working version with sample documents
3. **Evaluate** – Measure accuracy against ground truth

4. **Iterate** – Tune configurations, add repair rules, improve aliases until fill rate plateaus
5. **Integrate** – Connect the sprint output to the next sub-problem

5.3 Development Phases

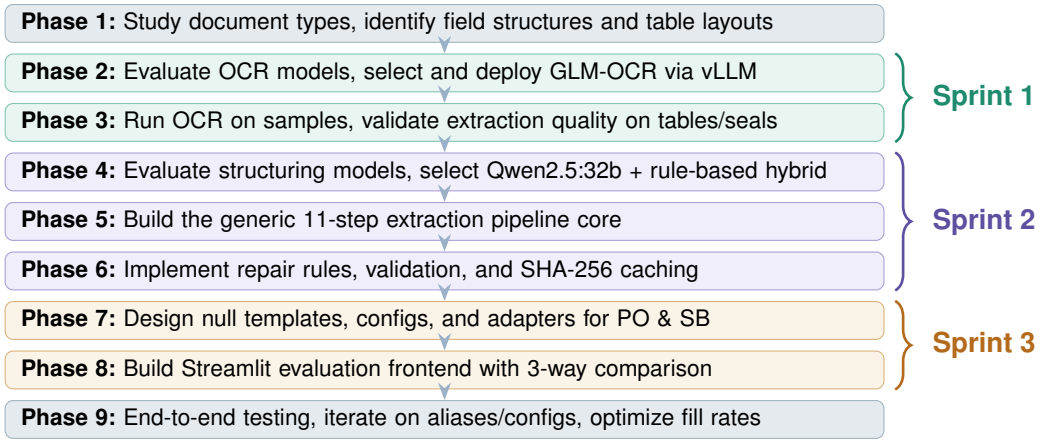


Figure 5.2: Development phases mapped to sprints

System Requirements

6.1 Hardware Requirements

Component	Minimum	Recommended
GPU	8 GB VRAM	24 GB+ VRAM (A100 / RTX 4090)
RAM	16 GB	32 GB+
Storage	50 GB free	100 GB+ (models + cache)
CPU	4 cores	8+ cores

Table 6.1: Hardware requirements

6.2 Software Requirements

Software	Version	Purpose
Python	3.12+	Core runtime for all components
UV	Latest	Fast package manager for GLM-OCR SDK
Ollama	Latest	Local LLM server hosting Qwen2.5
vLLM	Nightly	High-performance model serving for GLM-OCR
GLM-OCR	0.9B BF16	OCR model ($\sim$1.8 GB download)
Qwen2.5:32b	32B params	AI extraction model ($\sim$20 GB download)
PP-DocLayout-V3	Latest	Layout detection (25 label categories)
Streamlit	Latest	Evaluation frontend web framework
BeautifulSoup4	Latest	HTML table parsing in the pipeline
openpyxl	Latest	Color-coded Excel report generation
pandas	Latest	Data manipulation for evaluation

Table 6.2: Software and model requirements

Functional Requirements

ID	Requirement	Description
FR-01	PDF/Image Input	The system shall accept PDF files and image files (PNG, JPEG) as input documents.
FR-02	Layout Detection	The system shall detect and classify visual regions on each page—text blocks, tables, titles, formulas, seals, and images.
FR-03	OCR Extraction	The system shall extract text content from each detected region and produce per-page Markdown and JSON outputs.
FR-04	Result Merging	The system shall merge per-page OCR outputs into a single Markdown file and a single JSON file per document.
FR-05	Schema-Aligned Output	The system shall produce structured JSON output that matches the null template shape exactly—no extra keys, no missing keys.
FR-06	AI-Based Extraction	The system shall extract scalar field values (names, dates, addresses) using a local LLM with evidence-based prompting.
FR-07	Rule-Based Extraction	The system shall extract structured table data using deterministic column-index parsing from HTML tables.
FR-08	Value Repair	The system shall apply post-extraction fix rules (whitespace trimming, date normalization, currency stripping, etc.).
FR-09	Validation	The system shall validate extracted values for type correctness and required-field presence, producing a validation report.
FR-10	Response Caching	The system shall cache AI responses using SHA-256 keys to avoid redundant model calls on re-runs.
FR-11	Overlap Recall	The evaluation frontend shall compute per-page and overall field-level overlap recall for any prediction file.
FR-12	Alignment Report	The evaluation frontend shall generate downloadable, color-coded Excel files showing field-by-field GT vs Pred comparison.
FR-13	3-Way Comparison	The evaluation frontend shall support side-by-side comparison of Ground Truth vs MyModel vs GPT in a single Excel report.
FR-14	Plugin Registration	The system shall allow new document types to be added via config, adapter, and template files without modifying core pipeline code.

Table 7.1: Functional requirements

Non-Functional Requirements

ID	Requirement	Description
NFR-01	**Privacy**	All processing must run locally. No document data shall be transmitted to any external server or cloud API at any stage of the pipeline.
NFR-02	**Reproducibility**	Given the same inputs and cache state, the system shall produce bit-identical outputs. SHA-256 disk caching ensures deterministic AI responses.
NFR-03	**Extensibility**	Adding a new document type shall require only 3 new files and 1 registry line. Zero modifications to any core pipeline file.
NFR-04	**Accuracy**	Rule-based table extraction shall be 100% deterministic. AI-based extraction shall achieve $\geq$85% field-level match rate with repair rules enabled.
NFR-05	**Usability**	The evaluation frontend shall be operable by non-technical users with zero programming knowledge. All actions via upload, click, and download.
NFR-06	**Maintainability**	The codebase shall maintain strict separation between the generic core pipeline, document-specific plugins, and the evaluation layer.
NFR-07	**Portability**	The pipeline shall run on any Linux system with a CUDA-capable GPU. Model serving shall support Docker containerization.
NFR-08	**Observability**	Every pipeline run shall produce structured JSON logs with timestamps, field-level extraction status, and validation summaries.
NFR-09	**Scalability**	The system shall support horizontal scaling—processing N documents of the same type with a single schema and no per-document configuration.

Table 8.1: Non-functional requirements

Results

9.1 Shipping Bill Extraction

9.1.1 Our Pipeline (Qwen2.5:32b + Rule-Based)

Status	Count	Percentage
MATCH	199	85.04%
MISMATCH	22	9.40%
MISSING_KEY	11	4.70%
NEAR_MATCH	2	0.85%
Grand Total	**234**	**100.00%**

Table 9.1: Shipping Bill – Our Pipeline results (234 fields)

9.1.2 GPT-Based Extraction

Status	Count	Percentage
MATCH	204	86.44%
MISSING_KEY	23	9.75%
MISMATCH	9	3.81%
Grand Total	**236**	**100.00%**

Table 9.2: Shipping Bill – GPT-based extraction results (236 fields)

9.2 Purchase Order Extraction

9.2.1 Our Pipeline (Qwen2.5:32b + Rule-Based)

Status	Count	Percentage
MATCH	744	89.96%
MISMATCH	75	9.07%
MISSING_KEY	7	0.85%
NEAR_MATCH	1	0.12%
Grand Total	**827**	**100.00%**

Table 9.3: Purchase Order – Our Pipeline results (827 fields)

9.2.2 GPT-Based Extraction

Status	Count	Percentage
MATCH	527	63.57%
MISMATCH	269	32.45%
MISSING_KEY	33	3.98%
Grand Total	**829**	**100.00%**

Table 9.4: Purchase Order – GPT-based extraction results (829 fields)

9.3 Comparative Analysis

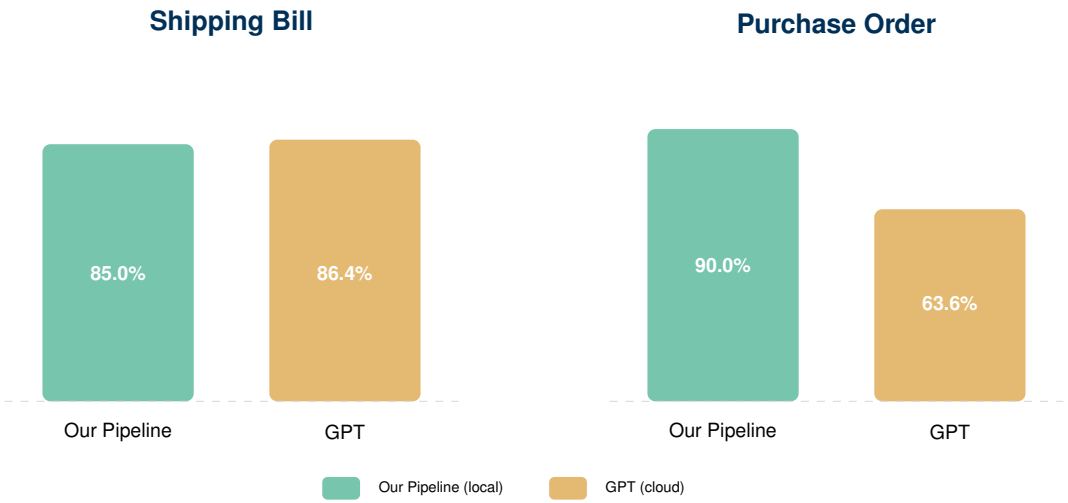


Figure 9.1: Match rate comparison – Our Pipeline (fully local) vs GPT (cloud-based)

Key Findings

- On **Purchase Orders**, the local pipeline outperforms GPT by **26.4 percentage points** (90.0% vs 63.6%). The rule-based table parser achieves 100% accuracy on structured multi-page tables where GPT frequently misaligns columns.
- On **Shipping Bills**, both approaches perform comparably (∼85–86%), with GPT showing slightly fewer mismatches but more missing keys.
- The local pipeline has **zero recurring cost** and **complete data privacy**, while GPT requires per-token cloud API charges and document upload to external servers.
- **NEAR_MATCH** cases (0.1–0.9%) are typically formatting differences (e.g., date format variations) that could be resolved with additional repair rules.

Scalability Report

The pipeline is designed for **horizontal scalability**—once a schema is defined for a document type, it can process an unlimited number of documents of that type with zero additional configuration.

10.1 Horizontal Scaling – One Schema, N Documents

The core design principle: **define once, run everywhere**. A single schema (null template + config + adapter) serves as the blueprint for an entire document type. Whether you process 1 document or 10,000 documents, the setup is identical. Each document is processed independently with no shared state.

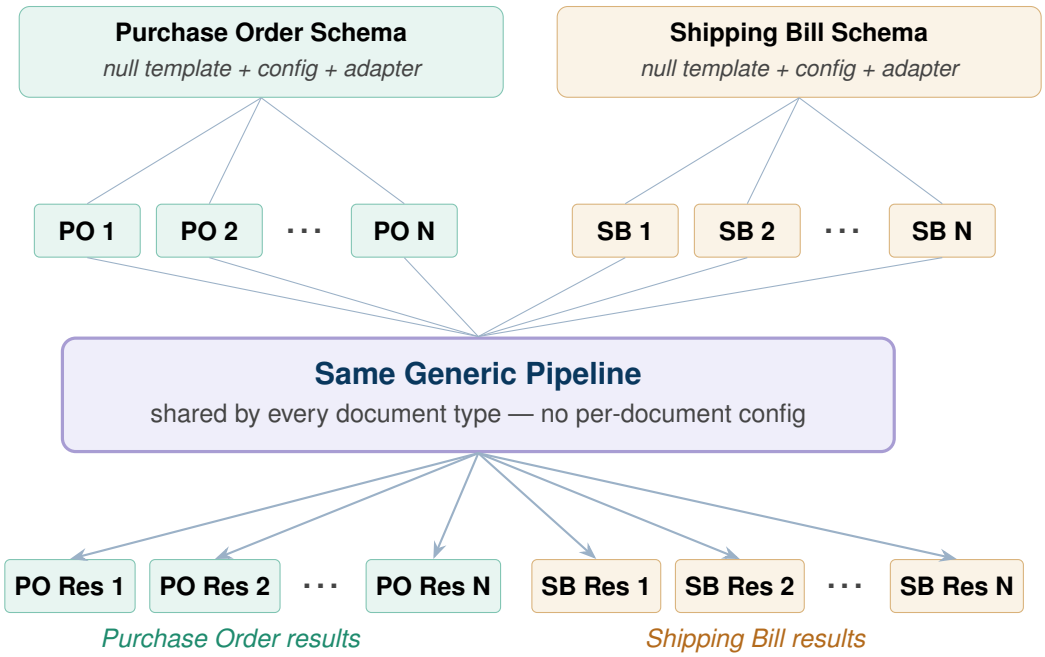


Figure 10.1: Horizontal scaling – one schema serves unlimited documents through the same pipeline

10.2 Key Characteristics Enabling Horizontal Scale

- **Stateless Processing** – Each document run is fully independent. No database, no shared memory, no inter-document dependencies. Documents can be processed concurrently across multiple machines or sequentially on a single machine.
- **No Per-Document Configuration** – All documents of the same type share a single schema, config, and adapter. Processing 1 document or 10,000 documents requires exactly the same one-time setup.
- **Disk-Backed Caching** – SHA-256 response caching means identical AI calls are never repeated. When processing a batch of similar documents, cache hit rates increase after the first few runs, effectively reducing compute cost to near-zero for repeated extraction patterns.
- **Batch-Ready** – A simple scripting loop over `run.py` processes an entire folder of documents. No orchestration framework, no job queue, no distributed system needed.
- **Multi-Machine Deployment** – The OCR server (vLLM) and AI server (Ollama) run as independent services on separate ports. They can be deployed on different machines, enabling GPU resource pooling across teams without any pipeline code changes.
- **Docker-Ready** – vLLM supports containerized deployment, enabling orchestration via Kubernetes or Docker Compose for enterprise-scale batch processing.

Advantages of the Project

1. **Complete Data Privacy** – All processing happens on local hardware. No document ever leaves the machine. This is critical for confidential commercial documents containing pricing, supplier terms, and trade secrets. Fully compliant with enterprise data governance policies.
2. **Zero Recurring Cost** – No API fees, no per-page charges, no subscription costs. Once the models are downloaded ($\sim$22 GB total), the pipeline runs for free indefinitely.
3. **Deterministic Table Extraction** – Rule-based parsing of structured tables is 100% accurate and reproducible. Unlike AI-based approaches that can hallucinate or misalign columns, column-index parsing always returns the correct value.
4. **Outperforms GPT on Complex Tables** – On Purchase Orders with multi-page table layouts, the local pipeline achieves 90% match rate vs GPT's 63.6%—a 26-point improvement. This demonstrates that domain-specific rules beat general-purpose AI for structured data.
5. **User-Friendly Evaluation** – The Streamlit frontend requires zero programming knowledge. Users upload three files, click buttons, and download color-coded Excel reports. The entire evaluation workflow is visual and intuitive.
6. **Plugin Architecture** – Adding a new document type requires zero changes to any core file. Three new files (template, config, adapter) and one registry line. The pipeline is future-proof and maintainable by any developer.
7. **Intelligent Caching** – SHA-256 disk-backed response cache means identical extractions are never repeated. Re-runs for debugging, testing, and iteration are near-instant ($<$5 seconds).
8. **Comprehensive Quality Reports** – The color-coded Excel reports with MATCH (green), NEAR_MATCH (orange), MISMATCH (yellow), and MISSING_KEY (red) make it immediately obvious where the pipeline succeeds and where it needs improvement—no manual inspection required.

Conclusion

This project demonstrates that a fully local, open-source pipeline can **match or exceed cloud-based AI services** for structured data extraction from commercial PDF documents—while offering complete data privacy and zero recurring cost.

The key insight is the **hybrid extraction strategy**: using AI (Qwen2.5:32b) for free-text fields where natural language understanding is needed, and rule-based column-index parsing for structured tables where deterministic accuracy matters. This combination achieves 85–90% overall match rates while maintaining 100% accuracy on table sections. On complex purchase orders, the local pipeline outperforms GPT by 26 percentage points.

The **plugin-based architecture** ensures the system scales to new document types without touching any core pipeline code—adding a new type is a matter of hours of manual document study, not weeks of model training or data annotation.

The **user-friendly Streamlit evaluation frontend** closes the feedback loop, allowing non-technical users to visually assess extraction quality through color-coded Excel reports with field-level MATCH, MISMATCH, NEAR_MATCH, and MISSING_KEY status tracking.

The pipeline runs entirely on local hardware with no API keys, no cloud dependency, and zero recurring cost, making it suitable for processing confidential commercial documents at scale within enterprise environments at Bosch Global Software Technologies.

Bosch Global Software Technologies

ERD Intern 2026